AirVintage Technical Documentation

Mobile Responsiveness, Styling Verification, & Architectural Diagrams

---

AirVintage UI Responsiveness & Theme Contrast Validation

This walkthrough documents the comprehensive audit and successful validation of the mobile responsiveness, light/dark theme contrast, and cross-device network stability across the AirVintage platform.

---

1. Visual Verification & Styling Improvements

■■ Map Page Theme Overrides & Legibility

- **High-Contrast Dynamic Colors**: Created the helper `getAqiTextColor(aqi, isLight)` in AQIMap.jsx to calculate safe colors for text overlays. In Light Mode, yellow/gold (`#eab308`) is automatically darkened to a rich dark gold/yellow (`#ca8a04`), ensuring clear contrast on white floating controls.
- **Theme-Aware Legend**: Custom CSS rules were added to App.css to style the collapsible `.map-legend-pill` and `.map-legend-card` dynamically. In Light Mode, they shift from dark semi-transparent glassmorphism to a crisp, drop-shadowed white card with dark text, guaranteeing readability.
- **Responsive Control Positioning**: Confirmed that controls wrap cleanly and do not overlap on viewports down to 360px.

---

2. 7-Day Forecast Grid Responsiveness

- **Mobile Scroll Snapping**: Enabled the `forecast-day-grid` class on the Detailed Forecast page in ForecastSection.jsx.
- **Subtle Scroll Gradient**: Integrated a smooth right-edge fade mask inside the mobile media queries in App.css:

```
.forecast-day-grid { -webkit-mask-image: linear-gradient(to right, #000 85%, transparent 100%); mask-image: linear-gradient(to right, #000 85%, transparent 100%); }
```

This signals scrollability on mobile without cluttering the screen.

- **Dashboard Extended Forecast Cards**: Styled the mini-cards on the main dashboard to display as structured cards with hover transitions and theme-adaptive text sizes.

---

3. Cross-Device Connectivity

- **Host Resolution**: Re-verified that all React API fetches dynamically target `window.location.hostname` instead of a hardcoded loopback address, enabling other devices on the same local network to load dashboard data.
- **CORS Compliance**: Backend dynamically adds the host IP to the CORS allowed origins, ensuring zero fetch blocks during testing.

---

4. Interaction Validation Recording

A recorded browser flow demonstrating the correct loading, theme toggling, and mobile rendering can be viewed here:

[Image: Detailed Forecast Validation Video]

System Architecture & UML Diagrams

AirVintage System Diagrams

This document contains three architectural diagrams (Use Case, Activity, and Sequence diagrams) that illustrate the core workflows, system boundaries, and component interactions of the AirVintage platform.

---

1. Use Case Diagram

The Use Case diagram outlines the core features available to users and how they trigger integrations with the FastAPI backend, ML model, and external API providers.

```
graph TB %% Actors User([User]) SystemBackend[[FastAPI Backend]] OMAPI[Open-Meteo API] MLModel[Machine Learning Model] subgraph AirVintage System UC1(Detect Geolocation) UC2(Search City Manually) UC3(View AQI & Weather Dashboard) UC4(View Interactive AQI Map) UC5(Toggle Map Overlays & Layers) UC6(Explore 7-Day Forecast & Charts) UC7(Receive AI Health Recommendations) end %% Interactions User --> UC1 User --> UC2 User --> UC3 User --> UC4 User --> UC5 User --> UC6 User --> UC7 UC1 --> SystemBackend UC2 --> SystemBackend UC3 --> SystemBackend UC4 --> SystemBackend UC6 --> SystemBackend UC7 --> SystemBackend SystemBackend --> OMAPI SystemBackend --> MLModel
```

---

2. Activity Diagram

The Activity Diagram represents the operational workflow of a user session—from initiating location tracking to viewing predictions and exploring charts.

```
stateDiagram-v2 [*] --> StartPage : Load App StartPage --> LocationDetection : Detect Location? state LocationDetection { [*] --> CheckGPS CheckGPS --> UseGPS : Success (Raw Lat/Lon) CheckGPS --> ManualSearch : Reject / Error ManualSearch --> EnterCityName EnterCityName --> GeocodeLocation : Fetch Coordinates } LocationDetection --> FetchDashboardData : Coordinates Resolved state FetchDashboardData { [*] --> RequestAQIAndWeather RequestAQIAndWeather --> FetchExternalSensors : Get Open-Meteo Raw Data FetchExternalSensors --> RunMLCorrection : Feed Raw Data to FastAPI ML Model RunMLCorrection --> GenerateFinalAQI : Calculate Calibrated AQI & Recommendations } FetchDashboardData --> DisplayDashboard : Render Main UI state DisplayDashboard { [*] --> ShowHeroAQI ShowHeroAQI --> ViewDetailedForecast : User clicks Extended Forecast ShowHeroAQI --> ViewInteractiveMap : User clicks Map View } ViewDetailedForecast --> TabNavigation : Toggle hourly weather indices ViewInteractiveMap --> LayerToggle : Select base map / AQI & Temp overlays TabNavigation --> [*] : Exit LayerToggle --> [*] : Exit
```

---

3. Sequence Diagram

This Sequence Diagram traces the end-to-end messaging pattern for rendering the dashboard and running the machine learning-corrected AQI estimation.

```
sequenceDiagram autonumber actor User participant Frontend as React Frontend (App.js) participant Backend as FastAPI Backend (main.py) participant ML as ML Model (Predictor) participant API as Open-Meteo API User->>Frontend: Open Dashboard Frontend->>Frontend: Get GPS coordinates or City input Frontend->>Backend: POST /predict {lat, lon} activate Backend Backend->>API: Fetch raw air quality & weather parameters activate API API-->>Backend: Return raw AQI, PM2.5, humidity, wind deactivate API Backend->>ML: Pass parameters to ML Classifier activate ML ML-->>Backend: Return corrected/calibrated AQI and status deactivate ML Backend-->>Frontend: Return response {aqi, pm2_5, status, weather_summary} deactivate Backend Frontend->>Frontend: Render UI (High-contrast theme-aware text & badges) Frontend-->>User: Display Dashboard details (Calibrated AQI & Weather)
```